# 창고 운영을, 수량에서 업무 흐름으로

고객사별 창고 운영관리 SaaS · 재고 정합성 · 작업 추적 · 운영 관제

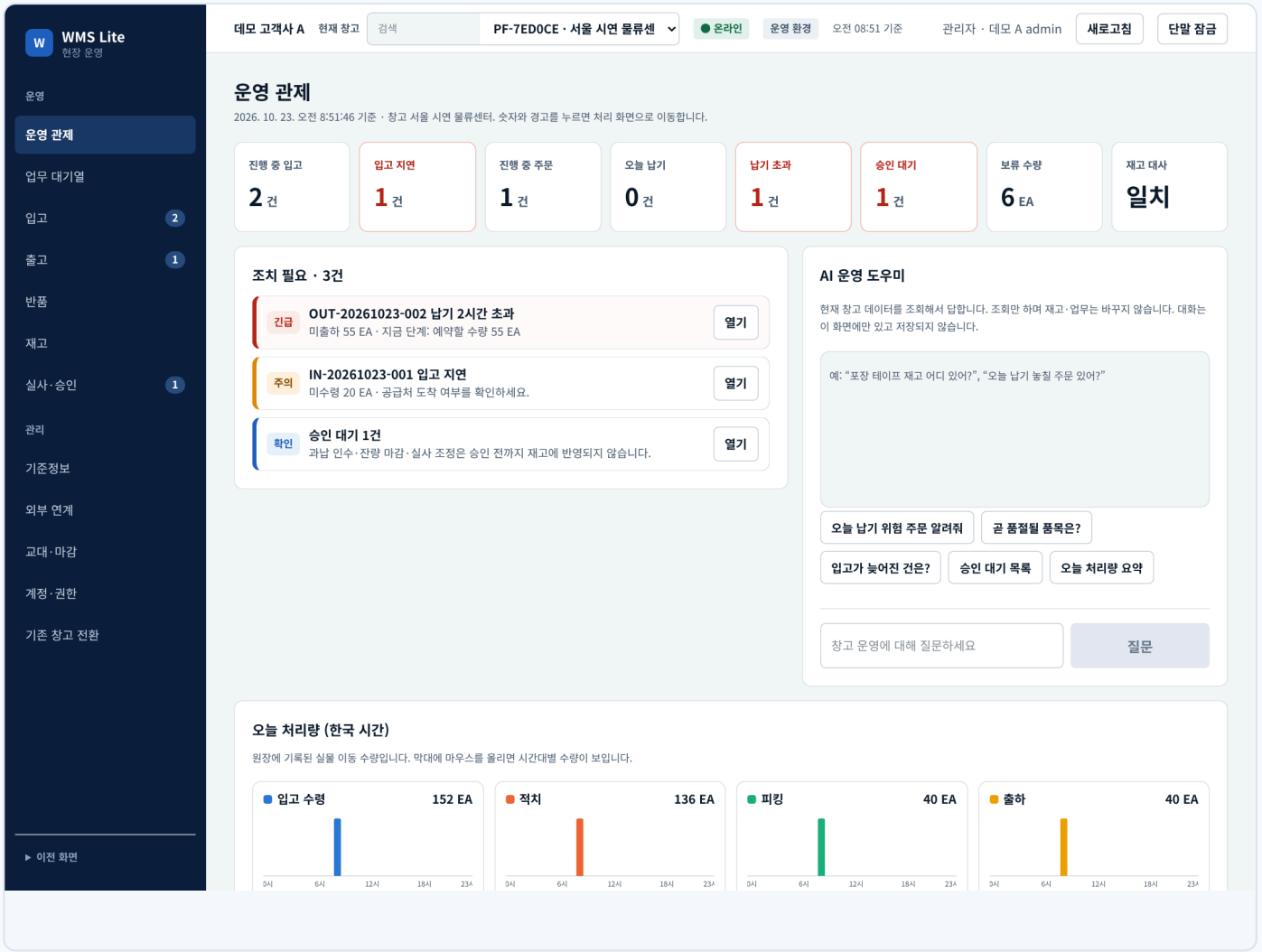
# WMS Lite.

**입고부터 출하·반품까지.**
**실물, 잔량, 담당자를 연결한**
**웹 기반 창고 운영 시스템.**

1인 프로젝트 | AI 도구 활용 | 공개 데모

**작성자 전우석 · 개발 기간 2일**

기획·업무 규칙 정의·개발 조율·운영 확인
Codex · Claude Code를 활용한 구현과 검증




최신 소스 11e2095 · 로컬 실행 · 합성 데이터의 운영 관제 화면

# “재고가 있다”와 “출고할 수 있다”는 다릅니다

물류 현장의 상태와 예외를 시스템의 처리 조건으로 구체화했습니다.

### 도착했지만 검수 전인 물건

수령·검수·적치를 나누고, 정상 적치 전에는 가용 재고로 사용하지 않습니다.

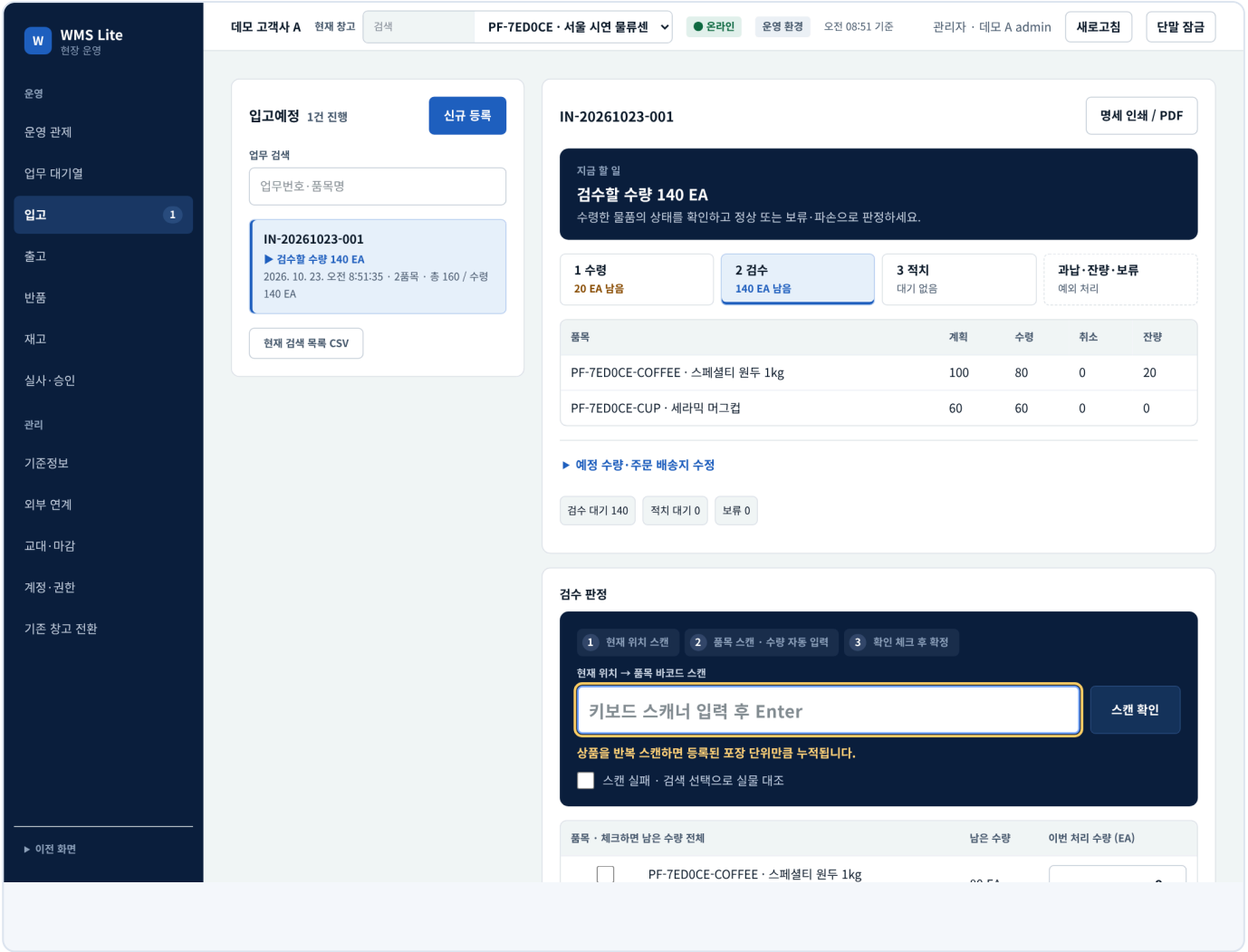
### 일부만 처리하고 교대하는 업무

완료 실적과 잔량, 다음 담당자를 같은 업무에 연결합니다.

### 재시도와 잘못된 수량 변경

중복 반영을 막고 원업무·작업자·사유가 남는 원장으로 추적합니다.

> 주요 범위: 기준정보 · 입고 · 출고 · 반품 · 실사·승인 · 교대 · CSV · 외부 연계



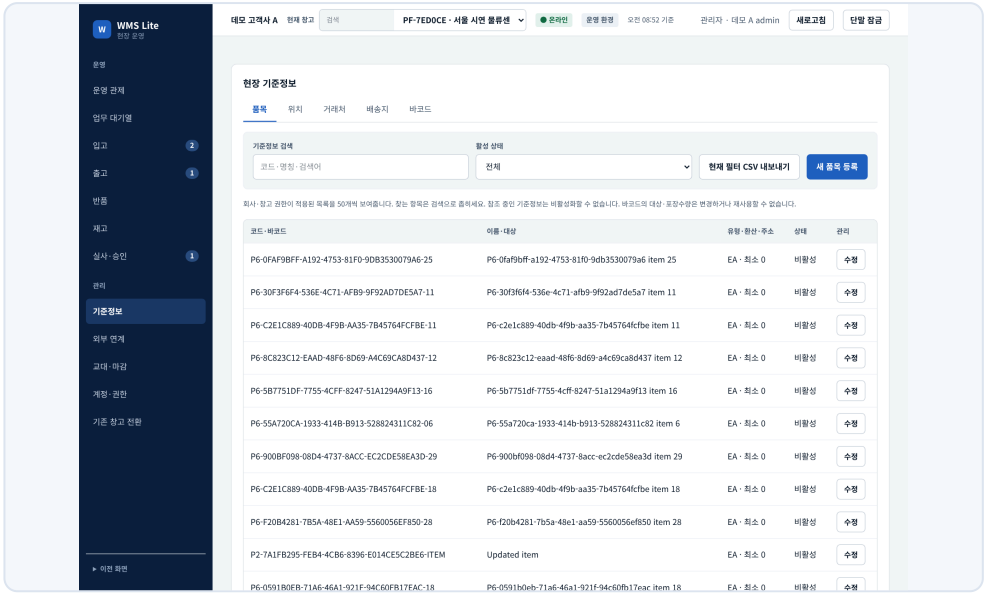

입고 160 EA 중 140 EA 수령. 미수령 잔량과 검수 대기를 구분하는 실제 화면

# 권한 확인과 업무 규칙을 서버에 모았습니다

Next.js · React · TypeScript · PostgreSQL · Better Auth · Docker




설명용 구조도. 외부 연계도 동일한 회사·창고·권한 통제를 거칩니다.




품목·위치·거래처·배송지·바코드를 관리하는 기준정보 화면

| 계층 | 선택과 목적 |
|---|---|
| 웹·API | Next.js App Router와 TypeScript |
| DB | PostgreSQL · 트랜잭션 · RLS · SQL 마이그레이션 |
| 검증 | Vitest · PostgreSQL 통합 · Playwright |
| 데모 | Vercel + Neon / 로컬 Docker 구성 별도 |

# 위험을 찾고, 처리 화면으로 이어갑니다

납기·지연·승인·보류·재고 대사를 한 화면에 모으고 현재 창고 데이터로 답합니다.

## 운영 관제

우선순위가 있는 조치 목록, 처리량, 작업자 실적, 납기 위험과 창고 배치도를 제공합니다.

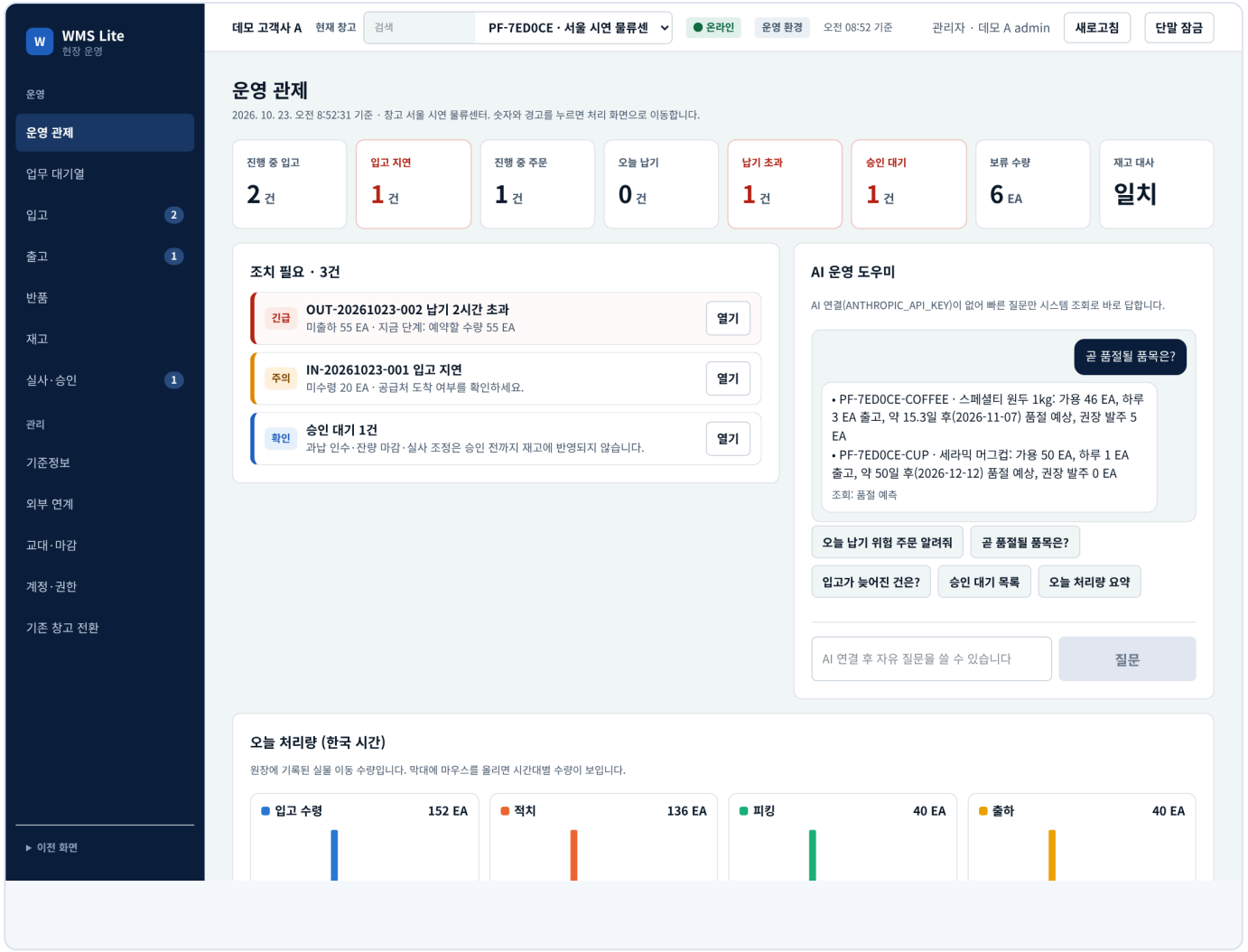
## 품절 예측

최근 7일·28일 출하량에 가중치를 적용하는 규칙 기반 계산입니다. 학습 모델의 정확도를 검증한 기능으로 표현하지 않습니다.

## 읽기 전용 도우미

Claude API 도구 호출 경로를 구현했습니다. 재고 변경 도구는 제공하지 않으며 회사·창고 권한 안에서 조회합니다.

> 오른쪽은 API 키 없이 빠른 질문이 조회 도구를 직접 실행한 결과입니다. 외부 모델 응답 시연이 아닙니다.




빠른 질문의 실제 조회 결과. 외부 AI API 호출·과금 없이 캡처했습니다.

# 지시 접수와 실물 반출입을 분리합니다

적용 제안 · 컨테이너·B/L·화주·TMS 실연계는 신규 설계 범위입니다.

핵심 통제

**TMS 접수 응답으로 재고를 바꾸지 않고, 실제 작업·게이트 실적을 원장에 연결합니다.**

### 컨테이너 위치

**재사용** 위치·이동 이력·스캔 대조

**추가** 컨테이너 ID·장치 위치 계층

### 반출입 지시 ↔ TMS

**재사용** 사건 버전·멱등 키·결과 조회

**추가** 지시·배차·게이트 실적 연결

### CFS 작업

**재사용** 부분 처리·검수·보류·승인

**추가** B/L·화주별 화물과 작업지시

## 우선 수락 시나리오

**01** **같은 반출입 지시를 두 번 수신**
실적·재고는 한 번만 반영

**02** **다른 컨테이너·위치를 스캔**
확정 차단, 원장 변화 없음

**03** **수정 전 지시가 늦게 도착**
버전 충돌 거절, 최신 지시 유지

**04** **부분 적출·적입을 반복**
B/L·화주별 실물·작업·원장 수량 일치

수입 예: 컨테이너 반입 → 적출·검수 → B/L별 장치 → 화물 반출. 수출은 화물 반입 → 혼재·적입 → 컨테이너 반출.
쿨스의 내부 명세를 확인한 구현 성과가 아닙니다. 실제 적용 시 고객사 절차·통관/보세 조건·TMS 계약을 확인합니다.

# 수령·검수·적치를 별도 실적으로 남깁니다

예정 등록만으로 재고를 만들지 않고, 실제 처리한 수량만 다음 단계로 넘깁니다.

### 입고예정
공급처·품목·예정 수량

### 수령
실제 도착 수량 → Q

### 검수
정상 적치 대기 / 품질 보류

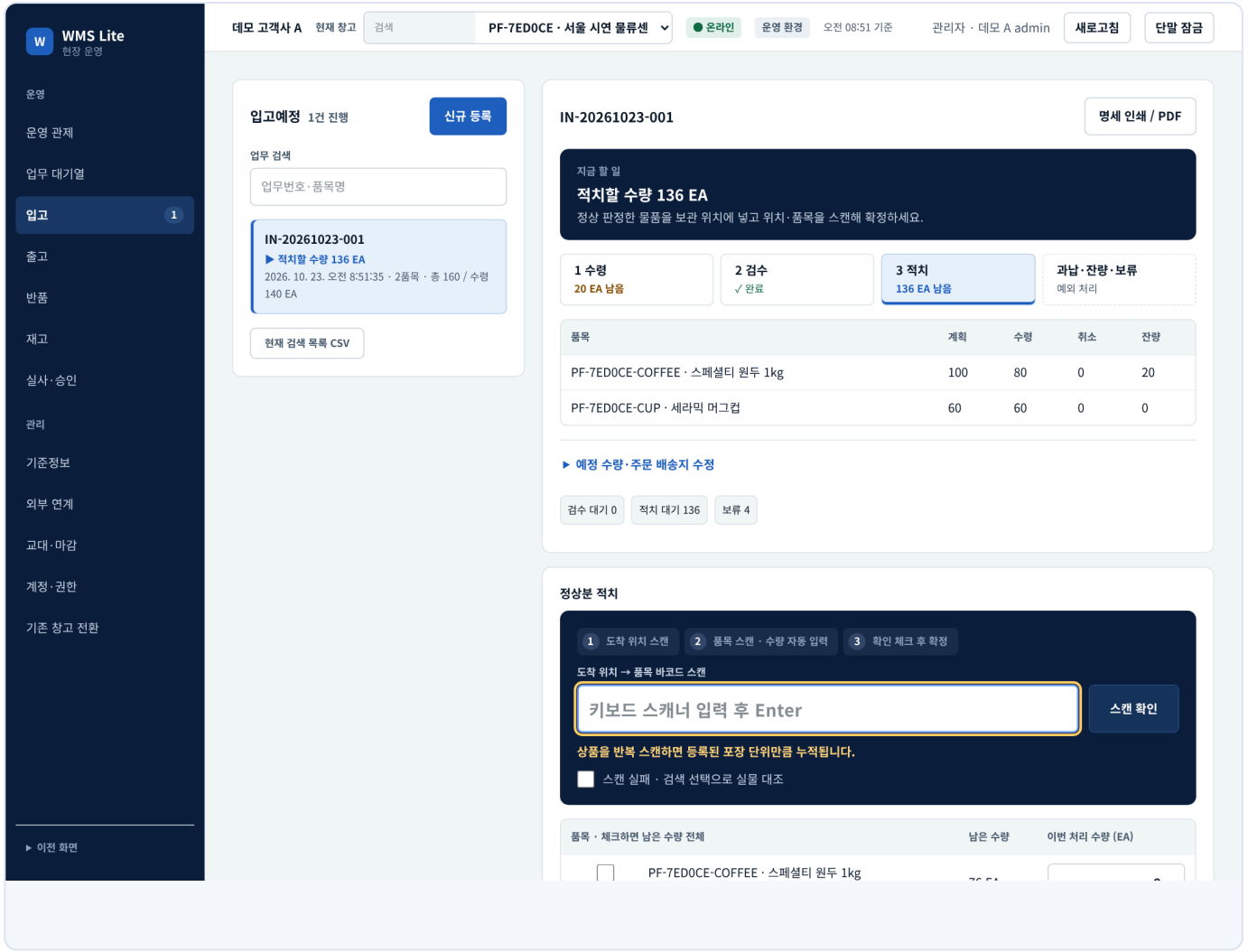
### 적치
보관 위치 확정 → S·A

## 부분 처리

미수령·검수 대기·적치 대기를 각각 유지합니다.

## 스캔과 대체 입력

품목·위치를 대조합니다. 검색 대체는 사유와 확인을 남깁니다.




정상분 적치 화면. 부분 처리 실적과 검수·보류 수량을 보존합니다.

# 예약한 위치에서 꺼내고, 포장한 물건을 출하합니다

주문 → 예약 → 피킹 → 포장 → 인계·출하. 단계 이동과 재고 반영을 구분합니다.

### 가용 재고 예약

전체 예약 또는 가능한 수량만 예약을 명시적으로 선택합니다.

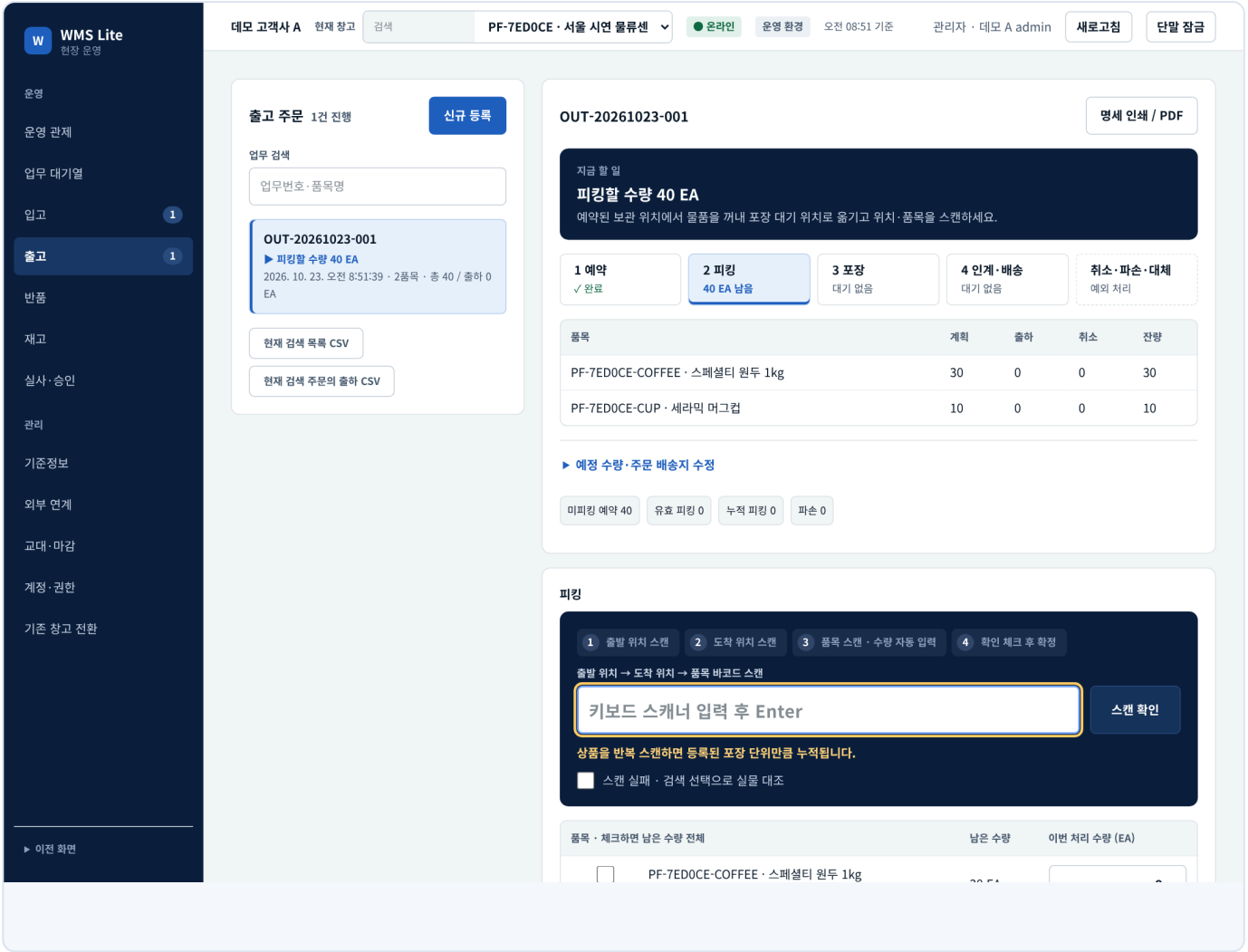
### 원행·위치 기반 피킹

예약 원행과 출발·도착 위치를 확인해 다른 주문의 물량과 구분합니다.

### 현재 단계와 남은 작업

처리 가능한 수량과 다음 행동을 안내하고, 완료된 실적은 유지합니다.

> 촬영 시나리오: 원두 30 EA + 머그컵 10 EA = 40 EA 출고




예약 40 EA 후 피킹 단계. 화면에 지금 할 일과 단계별 잔량을 표시합니다.

# 실제 인계 시점에 창고 재고를 차감합니다

배송지·포장·운송수단·인계자를 연결하고, 출하 이후 배송 사건을 기록합니다.

### 포장 전체 인계

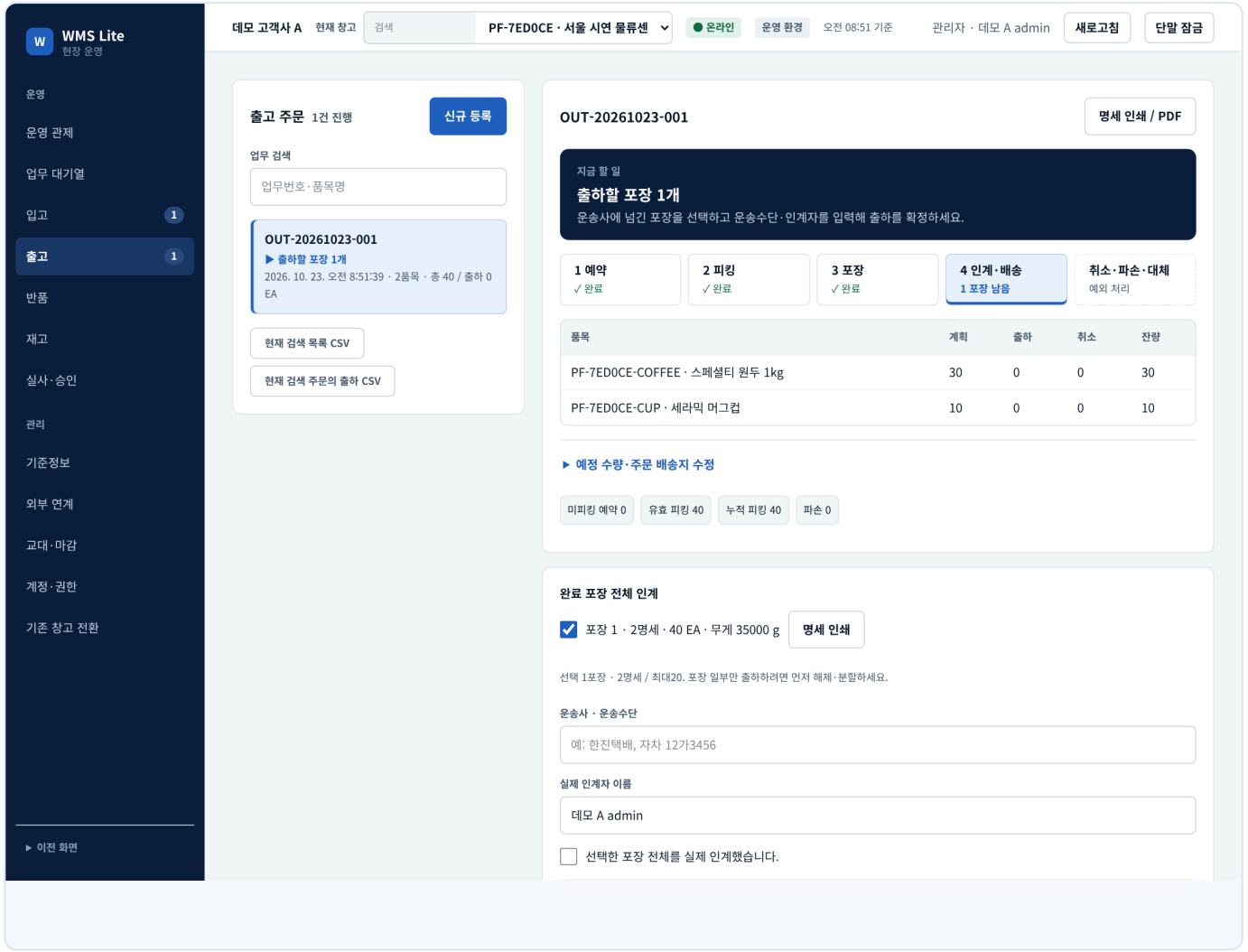
완료 포장을 단위로 출하합니다. 한 작업 최대 20명세를 적용합니다.

### 배송 이력

인계 완료·운송 중·완료·실패·분실을 기록하며, 정정은 사유와 이력을 남깁니다.

### 취소·파손·반품

미출하 취소는 실물 반납, 피킹 후 파손은 격리·대체 피킹, 출하 후 회수는 원출하 연결 반품으로 처리합니다.




원두·머그컵 포장 완료 후 실제 인계 화면. 배송지와 포장 내역을 확인합니다.

# 실물·가용·예약·보류를 일곱 지표로 구분합니다

H = S + P + Q + B · R은 S에 포함 · A는 예약과 위치 제한을 반영합니다.

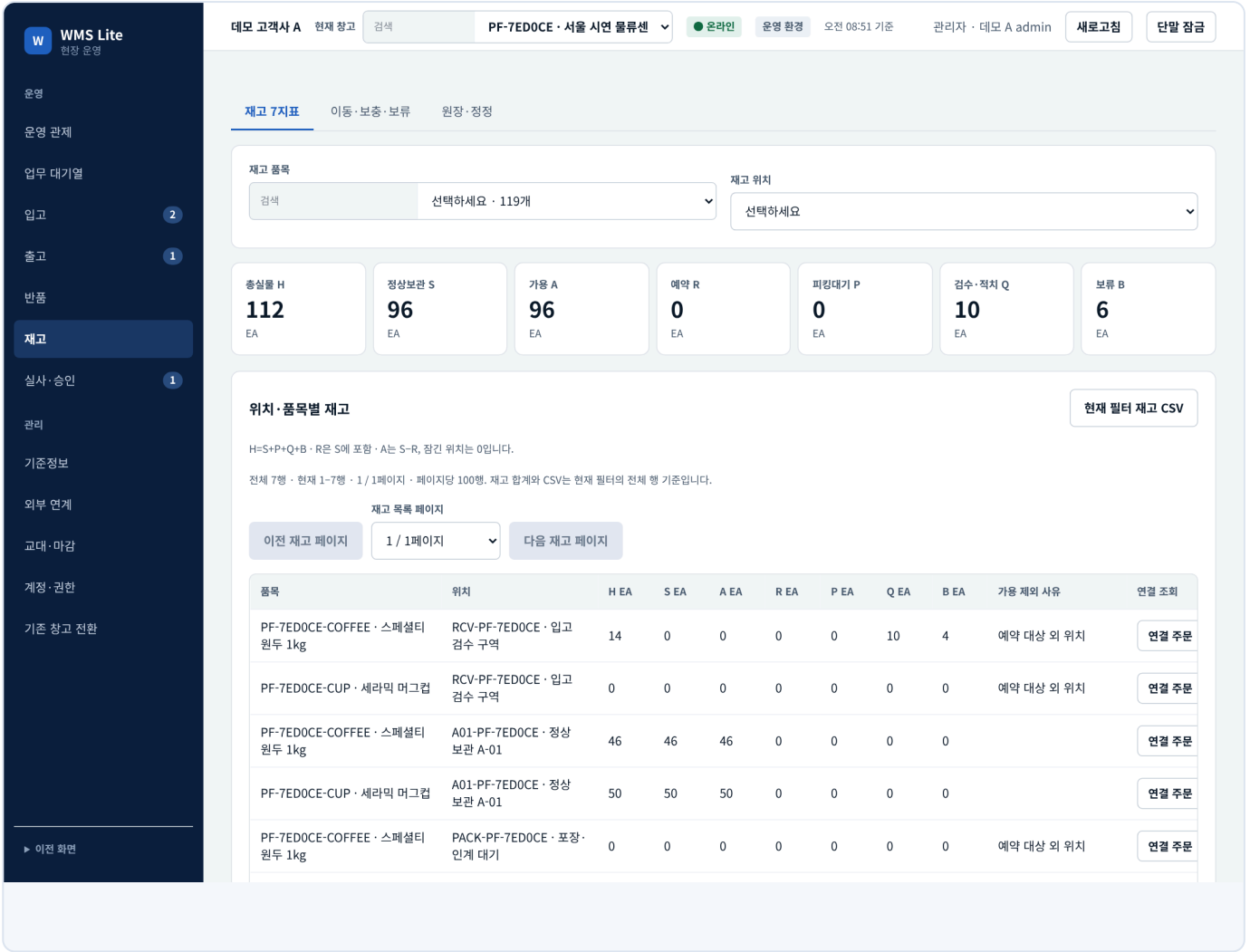
| | | | |
|---|---|---|---|
| **H** | 총실물 | **S** | 정상보관 |
| **A** | 가용 | **R** | 예약 |
| **P** | 피킹 | **Q** | 검수·적치 |
| **B** | 보류 | | |

## 이번 촬영의 실제 반영값

총실물 112 = 정상보관 96 + 검수·적치 10 + 보류 6 EA. 출하 40 EA를 반영한 결과입니다.

## 장부와 실물의 구분

실사·점검 잠금과 예약을 반영해 새 주문에 사용할 수 있는 양을 계산합니다.




이번 로컬 촬영에서 실제 API·DB에 반영된 재고 7지표와 위치별 잔고

# 재시도는 같은 요청으로, 정정은 연결된 새 기록으로

수량을 덮어쓰거나 원본 이력을 지우는 대신 처리 결과와 원장을 보존합니다.

### 멱등 처리

원래 요청 키와 처리 결과를 저장해 응답 유실 후에도 같은 작업을 중복 반영하지 않습니다.

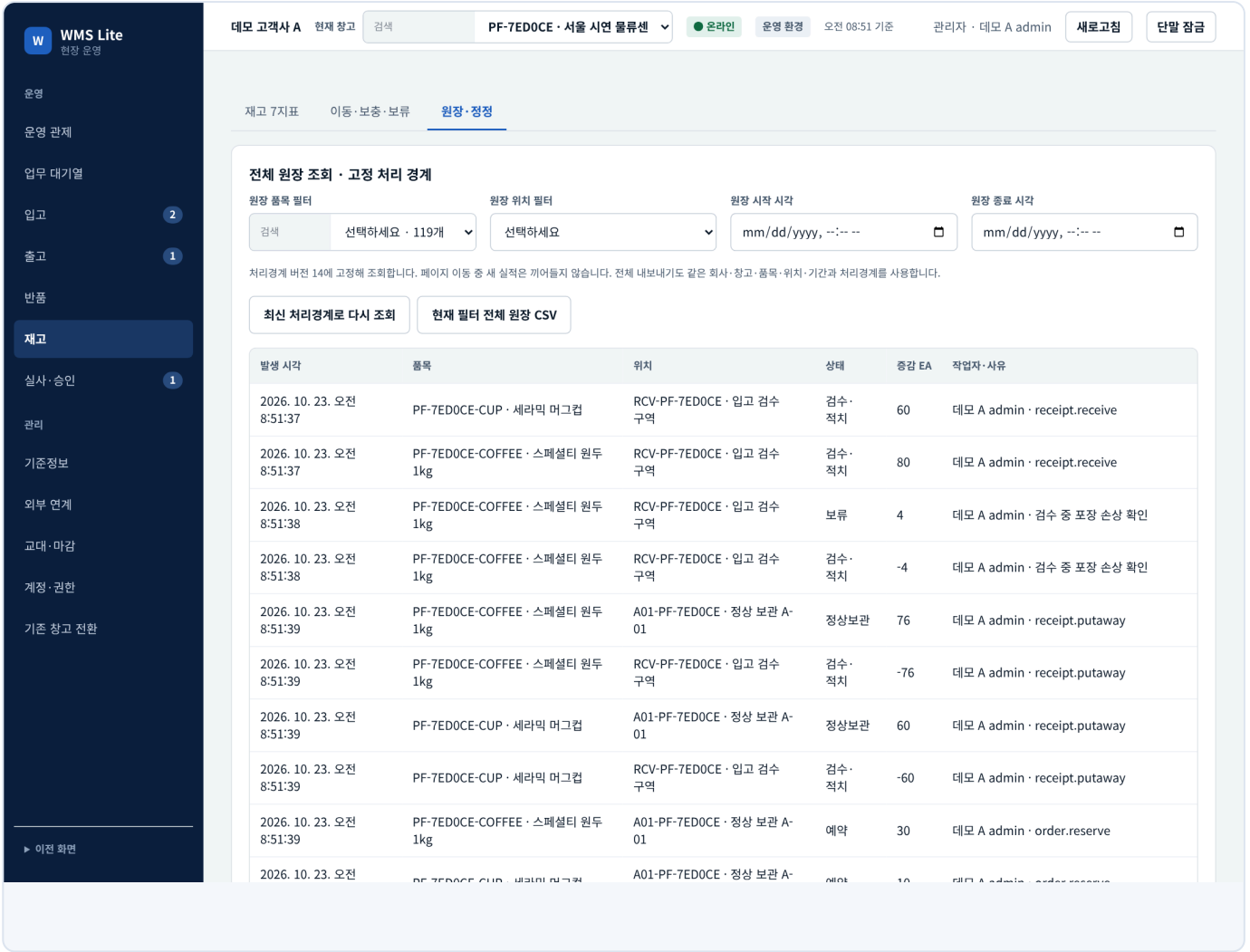
### 원자적 재고 변경

트랜잭션·잠금·버전 검사로 동시 작업을 제어하고, 실패 시 전체 요청을 롤백합니다.

### 원장과 대사

원업무의 수량 증감과 작업자를 연결합니다. 정정은 승인 후 보상 실적으로 기록합니다.

> MVP 공개 검증: 입고100 → 출고30 재시도 → 이동20 → 위치별50/20 및 원장 일치. 과거 실행의 검증 범위입니다.




현재 촬영 시나리오의 원장 조회. 품목·위치·처리 경계로 이력을 추적합니다.

# 조회 권한과 승인 자격을 별도로 검사합니다

여러 고객사가 같은 서비스를 사용하되, 작업자 본인의 실적을 스스로 승인하지 못하게 했습니다.

### 회사·창고·역할 경계

세션의 소속으로 회사 범위를 정하고 서버 권한 검사와 PostgreSQL RLS를 함께 적용합니다.

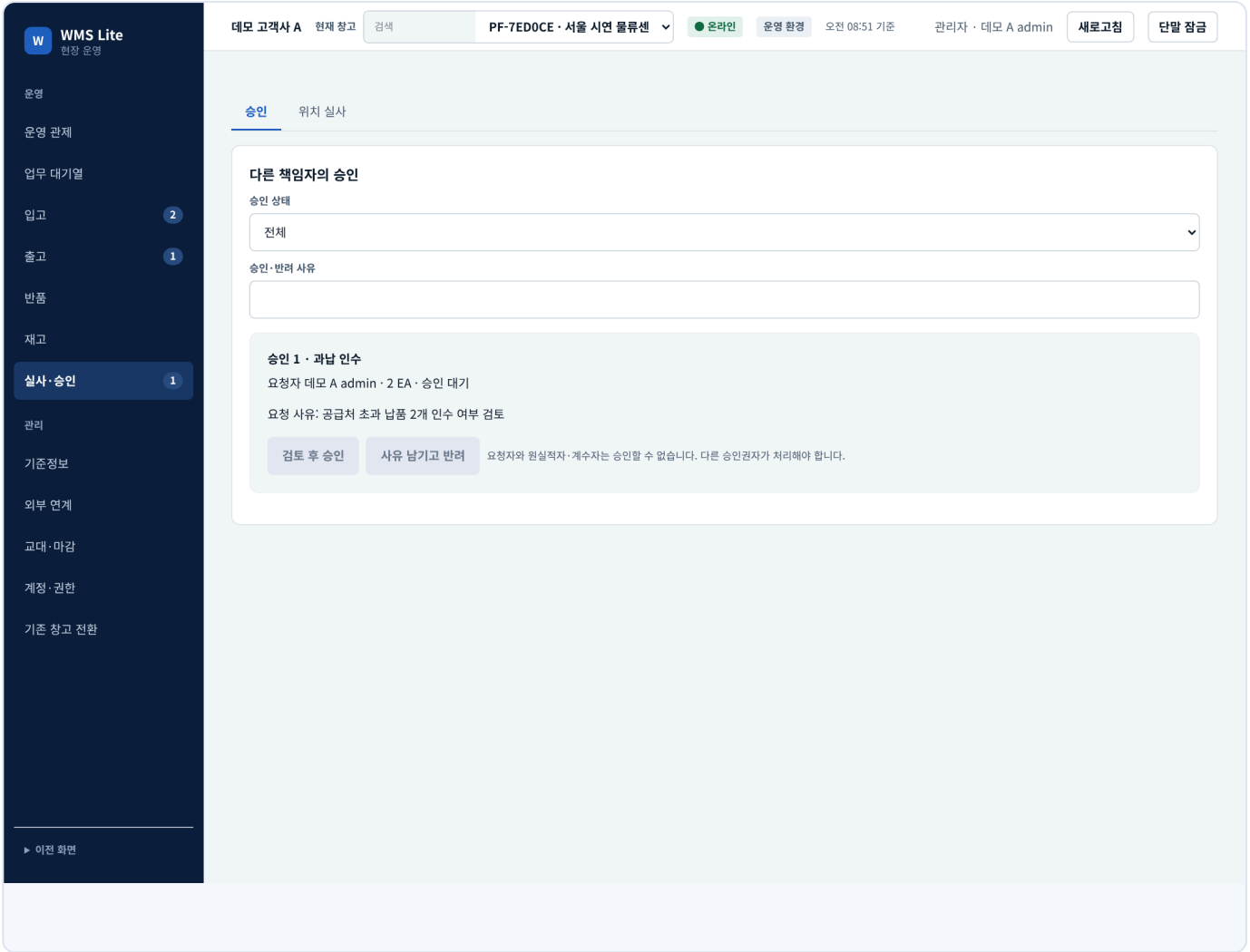
### 독립 승인

요청자·원실적자·계수자는 자기 승인에서 제외합니다. 같은 회사·해당 창고의 다른 적격 승인권자가 처리합니다.

### 과납의 실물 보존

초과 도착품은 보류로 기록합니다. 승인·반려 자체가 도착한 실물을 지우지 않습니다.

> 기존 공개 MVP에서는 A↔B 조회·전표 접근·취소 차단과 데이터 미변경을 검증했습니다.




본인이 요청한 과납 2 EA 승인. 승인 버튼이 비활성화되고 다른 승인권자 처리를 안내합니다.

# 명령별 결과를 확인합니다

아래 캡처는 재시도 0회의 정상 명령 흐름입니다. 이 화면으로 응답 유실·재시도 재현을 주장하지 않습니다.

**설정 가능한 패턴**
설정형 혼합
부족·파손·오스캔

**Seed / 공급**
20261020 / 10사건
수량 범위 1~100 EA

**캡처가 보여 주는 결과**
9개 명령 반영 확인
재시도 0회 · 정상 흐름

**판정 구분**
실행 종료 ≠ 전체 합격
전체 재고·원장 대사 별도

| 순번 | 사건 / 실물 작업 | 계정 | 상태 | 재시도 | 사유 |
|---|---|---|---|---|---|
| 1 | receipt.create | source | 반영 확인 | 0 | |
| 2 | receipt.receive | supervisor | 반영 확인 | 0 | |
| 3 | receipt.inspect | supervisor | 반영 확인 | 0 | |
| 4 | receipt.putaway | supervisor | 반영 확인 | 0 | |
| 5 | order.create | source | 반영 확인 | 0 | |
| 6 | order.reserve | supervisor | 반영 확인 | 0 | |
| 7 | order.pick | supervisor | 반영 확인 | 0 | |
| 8 | package.create | supervisor | 반영 확인 | 0 | |
| 9 | shipment.create | supervisor | 반영 확인 | 0 | |

명령 순서 확대: receipt.create → receive → inspect → putaway → order.create → reserve → pick → package.create → shipment.create. 계정·상태·재시도 열을 확인합니다.

## 왜 별도 앱인가

Node.js·SQLite에 입력·진행 상태를 보존하고 WMS API만 호출합니다. 위 표는 사건·작업 명령의 처리 순서와 계정·반영 상태를 확인하는 자료입니다.

## 무엇을 독립 검증하는가

WMS 계산 모듈을 재사용하지 않는 기대값으로 재고·전체 원장·원업무 수량을 대조합니다. 시험 회사 전용이며 현장 성능 검증을 대체하지 않습니다.

# 통과한 범위와 남은 기준을 구분했습니다

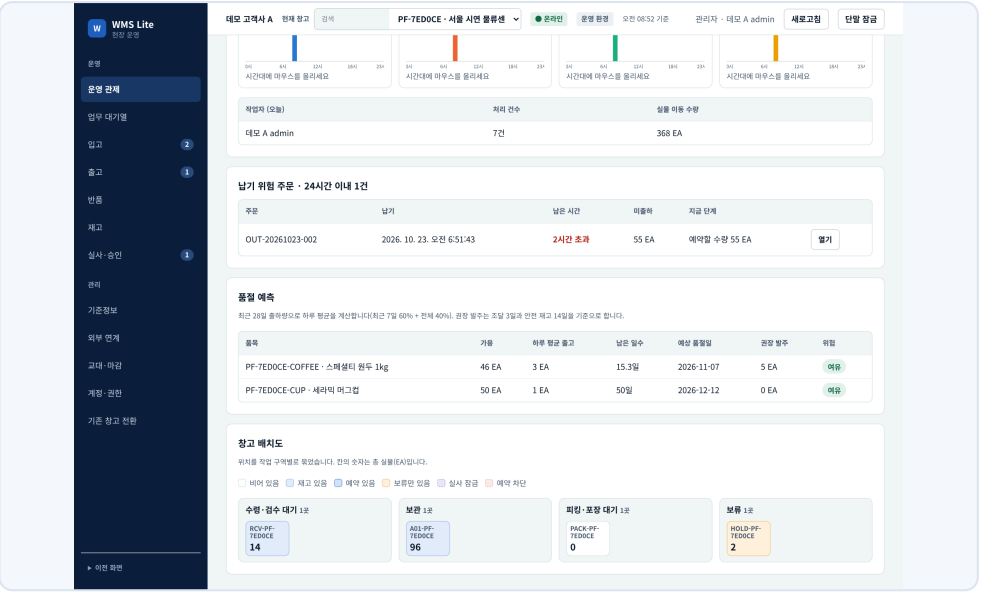
테스트 개수·화면 촬영·성능 시험·현장 운영을 하나의 완료 주장으로 합치지 않습니다.

**681**
기록된 이전 빌드의 로컬 검사

**22**
SQL 마이그레이션

| 검증 | 보존된 결과와 범위 |
|---|---|
| 단위 / 시뮬레이터 | 359 / 55개 통과 |
| PostgreSQL / 브라우저 | 223 / 44개 통과 |
| 기준 실행 | 2026-10-21 · 빌드 Lb3NKb1lCgfxlUs60Wy1T |
| 백업·복원 | schema22 로컬 복원·보존 기록 |
| 최신 추가 확인 | 관제·도우미 단위 검사 11개 통과<br>최신 소스의 로컬 실제 API·DB 화면 촬영 |

681은 자동화 테스트의 실행 사례 수이며 수작업 작성량을 뜻하지 않습니다. 2026-10-21 기준 기록으로, 최신 UI 변경 전체의 회귀 결과와 구분합니다.




현재 관제 화면의 품절 예측·배치도. 화면 수치와 성능 목표 합격은 별도입니다.

### 남은 검증

혼잡 시험은 목표 지연·완주 기준 미달입니다. 작업마다 전체 상태를 읽고 검증하는 비용과 동시 처리 대기가 병목 후보입니다. 단계별 읽기·대사·잠금 대기를 분리 계측한 뒤, 정합성 검사를 유지하며 조회 범위를 줄여 동일 부하로 재검증하겠습니다. 단일 원인 확정이나 개선 완료를 주장하지 않습니다.

실제 8시간 교대·실물 장비·현장 병행은 추가 검증 대상입니다. 합성 데이터 기반 프로토타입이며 상용 고객사 실운영 성과로 표시하지 않습니다.

# AI 결과물에서 업무 누락을 찾고, 규칙을 결정했습니다

개발 기간 2일 · AI는 구현·테스트를 지원했고, 업무 판단과 검토 사례를 구체적으로 남겼습니다.

직접 발견·수정 요구

## ① 출고인데 “도착: 없음”

**발견** 출고 확인창이 창고 위치만 보여 주어 받는 거래처와 실제 배송지를 확인할 수 없었습니다.

**판단** 내부 보관 위치와 외부 도착지를 분리하고, 반복 입력 대신 등록 배송지를 선택하도록 요구했습니다.

**구현·검증** 거래처·배송지를 필수화하고 서버에서도 누락을 거절합니다. 포장에 저장한 주소는 이후 주소록·주문 수정과 분리해 재출력합니다.

> **설계 근거** 주소록을 실시간 참조하면 과거 출하 명세까지 바뀔 수 있어, 당시 배송정보를 보존해야 합니다.

기획 피드백 검토·규칙 채택

## ② 100개 예정에 105개 도착

**충돌** 작업자의 과납을 모두 금지하면 실제 도착한 초과 물량을 전산에 남길 수 없습니다.

**판단** 작업자는 실물 105개를 기록하되 초과 5개는 보류합니다. 정상 인수 승인은 다른 적격 책임자에게 분리합니다.

**구현·검증** 승인 전 가용 전환을 차단하고, 반려해도 보류 실물을 삭제하지 않습니다. 승인이 나도 해제·검수·적치가 남습니다.

> **설계 근거** 실물의 존재와 사용할 권한은 별개입니다. 승인 상태가 총실물을 임의로 바꾸면 안 됩니다.

**본인 기여:** 문제 발견·업무 규칙 선택·수정 요구·운영 확인. **AI 지원:** 코드 작성·테스트 생성·문서화. 모든 코드를 수작업 작성했다고 표현하지 않습니다.

근거: 출고-배송정보-검증결과.md · 배송지-주소록-검증결과.md · 현장 운영 기획서 6~8장 · 본인 과납 요청의 자기 승인 차단 화면(11쪽).

# 같은 키로 재전송해도 재고를 한 번만 반영합니다

실제 코드 발췌 · executeWorkflowInTransaction · src/server/v2/workflow/index.ts

고정 소스 L228부터 발췌 · 줄바꿈 정리·중간 조회/반환 생략

```
await timeAsync("workflow.request_lock", () =>
  client.query(
    "SELECT pg_advisory_xact_lock(hashtextextended($1,0))",
    [identity.tenant.id + ":" + input.request_id],
  ),
);
// … 창고 상태 잠금 생략
const requestHash = timeSync(
  "workflow.request_hash", () => hash(input),
);

// … 저장 결과 조회 생략
if (saved.rowCount) {
  if (saved.rows[0].request_hash !== requestHash)
    fail("IDEMPOTENCY_CONFLICT");
  // … 원래 결과 재반환 분기 생략
}
```

**DB 제약** PRIMARY KEY(tenant_id, request_id)

**원자성** 재고·원장·요청 결과를 같은 트랜잭션에 저장한 뒤 COMMIT

업무 처리 코드 ↗ · withTenant의 COMMIT·ROLLBACK ↗

### ① 권한 확인 후 동일 키 잠금

회사 ID + request_id로 트랜잭션 advisory lock을 잡아 같은 요청의 동시 실행을 직렬화합니다.

### ② 키가 같아도 본문은 비교

정규화된 요청의 SHA-256을 저장값과 비교합니다. 내용이 달라지면 IDEMPOTENCY_CONFLICT로 거절합니다.

### ③ 이미 성공했다면 재실행하지 않음

저장된 작업 결과를 replayed: true로 반환합니다. 전체 응답의 재고 스냅샷은 현재 상태로 조회합니다.

### ④ 처음 온 요청이면 함께 커밋

새 재고·원장·요청 결과를 함께 저장합니다. COMMIT 후 응답이 끊겨도 같은 키로 결과를 다시 확인합니다.

**설계 의도:** 잠금은 동시 중복을, 해시는 키의 잘못된 재사용을, 트랜잭션은 재고와 처리 결과의 불일치를 막습니다. 요청 해시는 인증 수단이 아닙니다.

# 입고에서 출하까지, 원장으로 결과를 확인합니다

검토자가 따라갈 수 있는 시연 순서와 근거 문서를 함께 제공합니다.

**1** **회사·창고·권한**
작업 범위와 운영 관제 소개

**2** **입고**
수령 → 검수 → 적치와 Q·S·A 전환

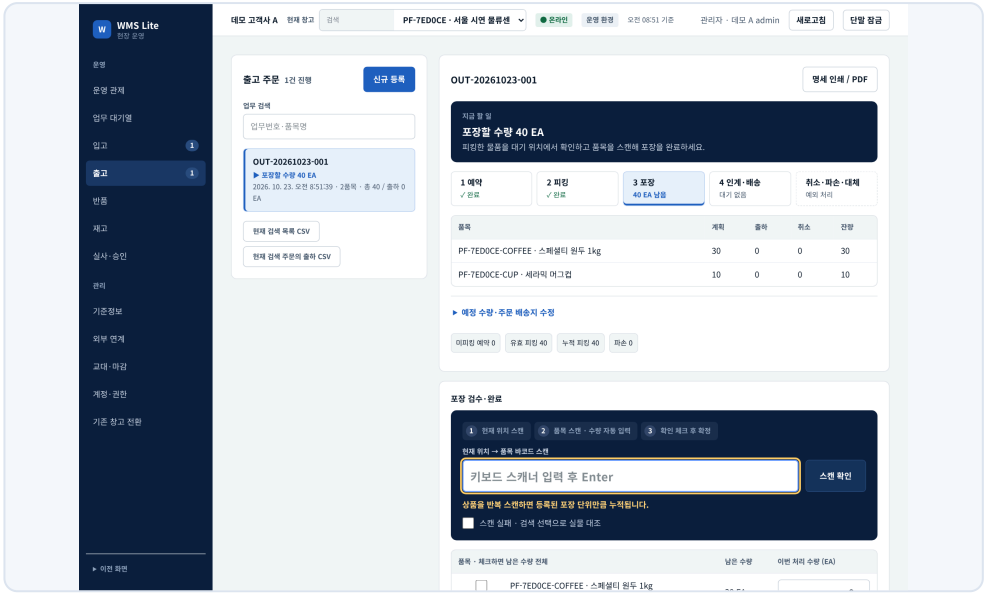
**3** **출고**
배송지 → 예약 → 피킹 → 포장 → 인계

**4** **예외·추적**
과납 보류·독립 승인, 재고·원장 연결

**5** **검증**
시뮬레이터·자동화 근거와 남은 성능 기준

GitHub 저장소 ↗ | 공개 데모 ↗

구현·검증 추적표 ↗ | 사용자 매뉴얼 ↗




포장 단계의 실제 화면. 시연은 별도 합성 데이터로 진행합니다.

**자료 범위**

구현 화면: 소스 11e2095 · 실제 로컬 API·PostgreSQL · 합성 데이터

시뮬레이터: 기존 로컬 시험 기록 발췌

상세 실행 근거는 저장소의 검증 문서에 보존했습니다.

작성자: 전우석 · 개발 기간: 2일.

계정 비밀번호·API 키·DB 접속정보는 포함하지 않았습니다.